A Synopsis Report

on

VocaSense: Voice Interaction and Coding Assistant

Submitted in Partial Fulfillment of the Requirements for the Degree

of

BACHELOR OF TECHNOLOGY

in

Department of Computer Engineering

by

Siddhi Deshmukh	**A59**
Arundhati Dhaygude	**A34**
Harshal Kolate	**A54**

Guided by

Dr. Sunita Nandgave

G H Raisoni College of Engineering and Management

Wagholi, Pune

(An Empowered Autonomous Institute, Affiliated to SPPU, Pune,

Affiliated by NAAC A+ Grade)

June, 2026

Synopsis

1. Introduction

In today's digital environment, presentations, online meetings, and live coding sessions are becoming an essential part of education and professional work. Most existing systems still rely on keyboards, mice, or handheld devices for navigation, which can interrupt the speaker's flow and reduce audience engagement. A hands-free approach can make communication more natural, efficient, and accessible.

Beyond single-command recognition, voice-driven systems that must handle continuous, multi-speaker dialogue, such as meeting recording and note generation, additionally require accurate speaker attribution over time. Recent work has shown that combining speaker diarization with ASR transcription and a constrained, locally-deployed language model for post-processing can reduce speaker-attribution errors and repetitive transcription artifacts, while avoiding the privacy risks of sending sensitive audio to proprietary cloud APIs [1]. This finding is particularly relevant to any system that automatically generates meeting notes, since raw ASR output attributed to the wrong speaker can significantly reduce the usefulness of a transcript.

Voice-driven interaction depends fundamentally on the reliability of the underlying speech technologies. Automatic speech recognition (ASR), the process of converting spoken audio into text, has evolved substantially over the last decade, moving from classical statistical approaches such as Hidden Markov Models and Gaussian Mixture Models toward deep-learning architectures, including recurrent networks, convolutional networks, transformers, and conformer-based models [2]. This evolution has directly improved the feasibility of real-time, voice-controlled applications, since modern end-to-end ASR pipelines can achieve substantially lower word error rates than earlier hybrid systems [2].

Beyond low-level speech and speaker processing, artificial intelligence is also being applied to the design of intelligent, feedback-driven assistants in educational and engineering contexts. A recent systematic review of AI-driven Intelligent Tutoring Systems (ITS) in engineering education found that such systems increasingly combine natural language processing, adaptive learning pathways, and real-time feedback to support learners working with abstract or technically complex material [3]. The same review explicitly identifies multimodal interaction, combining text, code, voice, and diagrams, as a promising but underexplored direction for future ITS platforms, and notes a persistent gap in systems that support both the learner and the instructor simultaneously [3]. This observation is directly relevant to an AI Presentation Assistant that must interpret spoken commands, slide content, and, in the coding-assistant mode, source code, in order to provide timely and pedagogically useful feedback to its user.

The coding-assistant component of a voice-driven platform additionally depends on the system's ability to represent and reason about source-code structure, rather than treating code

as unstructured text. Recent work on the Semantic Code Graph (SCG) demonstrates that a detailed, graph-based information model of code dependencies, capturing definitions, declarations, and the relationships between them, can improve software comprehension compared to conventional Class Collaboration Network and Call Graph representations, and can support practical tasks such as identifying important code entities and detecting similar or duplicated code [4]. Although this work was not designed for voice interaction, its underlying principle, that a structured representation of code dependencies enables more useful, context-aware assistance, is directly applicable to the AI Code Suggestions and AI Error Detection features envisioned for the coding module of the proposed platform.

However, speaker-recognition systems are known to degrade in performance when the input speech is emotionally inflected or acoustically variable, a limitation that has motivated recent work on data augmentation techniques such as synthetic emotional utterance generation to improve robustness [5].

A related but distinct challenge is identifying and verifying *who* is speaking, which is the domain of speaker recognition. Speaker identification, verification, and diarization techniques allow a system to distinguish a specific user's voice from other speakers or background interference [6], which is directly relevant to reliable command recognition in noisy, multi-user environments such as classrooms or open offices.

This project proposes an AI-powered voice interaction platform that enables users to control digital content using simple voice commands. The system allows users to upload and manage presentation files, navigate through content, open a coding workspace, and write code using speech-to-text technology. It also supports multilingual voice commands, making the platform more convenient for users with different language preferences. To improve reliability, AI-based noise filtering helps distinguish the speaker's voice from background sounds, resulting in more accurate command recognition.

In addition to voice interaction, the platform automatically records meetings and generates concise notes, reducing the effort required for manual documentation. These features make the system suitable for classrooms, technical demonstrations, online training sessions, corporate meetings, and accessibility-focused applications.

By combining voice recognition, artificial intelligence, speech-to-text conversion, and web technologies into a single platform, this project aims to provide a smarter, more interactive, and user-friendly solution for modern digital communication and collaboration. Unlike the works discussed above, which address speech recognition, speaker recognition, meeting transcription, intelligent tutoring, or code comprehension as isolated research problems, this project integrates these capabilities into a single, application-level voice interaction and coding assistant.

2. Literature Review

This section reviews the available literature relevant to the voice, speech, language-processing, tutoring-assistant, and code-comprehension components of the proposed system. The reviewed works are presented in descending order of publication year, from the most recent to the oldest, so that the discussion reflects the current state of the art before considering earlier foundational contributions.

Deonise et al. (2026) [1] propose an offline pipeline for speaker-attributed meeting transcription that integrates Pyannote-based speaker diarization, Whisper-based ASR, and a constrained, locally-deployed open-weight large language model (Qwen2.5, 3B and 7B variants) for transcript post-processing. The central research problem addressed is the "who said what" challenge: diarization alone only estimates speaker activity intervals, while ASR alone only produces text, and neither is sufficient on its own for a usable, speaker-attributed meeting transcript. Their methodology evaluates the pipeline on the AMI Meeting Corpus using Diarization Error Rate (DER), Jaccard Error Rate (JER), Purity, Coverage, Word Error Rate (WER), and concatenated minimum-permutation Word Error Rate (cpWER). A key finding is that unconstrained LLM rewriting of transcripts reduces repetitive ASR artifacts but degrades lexical fidelity (increasing WER/cpWER), whereas a conservative, validation-and-acceptance-filtered correction strategy, which only permits the LLM to reassign speaker labels rather than rewrite text, preserves WER/cpWER close to the unprocessed baseline while still reducing artifacts. The main limitation is that the approach is evaluated only offline and only on the Headset Mix audio condition of AMI; far-field microphone-array conditions and real-time/streaming operation are left as future work. This work is directly relevant to the meeting-recording and automatic note-generation feature of the proposed project, since it demonstrates a practical, privacy-preserving way to combine ASR with speaker attribution without relying on a proprietary cloud API.

Ahlawat, Aggarwal, and Gupta (2025) [2] present a systematic literature review of deep-learning-based ASR research conducted since 2010. The paper traces the evolution of ASR architectures from Gaussian Mixture Model/Hidden Markov Model (GMM/HMM) hybrid systems, through deep neural networks, recurrent neural networks (including LSTM/GRU variants), convolutional neural networks, to attention-based Transformer and Conformer architectures, along with recent large pretrained models such as Wav2Vec, Wav2Vec2, HuBERT, Whisper, and WavLM. The review also surveys monolingual and multilingual ASR datasets, toolkits (e.g., Kaldi, ESPnet, WeNet, NeMo), evaluation metrics (WER, CER, PER, BLEU, etc.), and discusses computational, ethical, and environmental considerations, as well as the impact of reinforcement learning on ASR. The authors report that state-of-the-art transformer/conformer models have pushed WER below 5% on benchmark English datasets, while low-resource languages still show substantially higher error rates, and multilingual and code-switching ASR remain open research problems. This survey is relevant to the proposed

project's speech-to-text coding module and multilingual voice-command support, since it identifies both the ASR architectures suitable for real-time voice interaction and the persistent accuracy gap for non-English or code-switched speech, which the project must account for when supporting Hindi and Marathi commands alongside English.

Rodrigues, Pinto, and Gonçalves (2025) [3] present a PRISMA-guided systematic literature review of 46 peer-reviewed studies on AI-driven Intelligent Tutoring Systems (ITS) in engineering education. The research objective is to synthesize how AI-based ITS deliver personalized instruction, adaptive feedback, and learner-progress monitoring in a domain where learners often struggle with abstract, technically complex content. The problem addressed is twofold: general pedagogical surveys do not focus on the discipline-specific demands of engineering education, and existing ITS research is fragmented across isolated feature areas rather than being synthesized around personalization, feedback, and monitoring together. Methodologically, the authors searched IEEE Xplore, Scopus, and SpringerLink, screened an initial 2,212 records down to 46 included studies, and classified the AI techniques used, including Natural Language Processing, machine learning, and federated learning, and the ITS features they support, such as adaptive content selection, real-time feedback (RTF), and adaptive content sequencing (ACS). A key contribution is the identification of a structural imbalance between *interaction-centric* approaches (reactive, student-facing personalization) and *model-centric* approaches (long-horizon planning and instructor-facing analytics), revealing a persistent gap in dual-user design where student adaptivity is not complemented by instructor-facing intelligence. The authors explicitly recommend that future ITS combine multimodal interaction, text, code, voice, and diagrams, to better represent the complexity of technical problem-solving. Limitations acknowledged by the authors include reliance on only three databases, English-language-only inclusion criteria, and a qualitative rather than quantitative (meta-analytic) synthesis. This review is relevant to the proposed AI Presentation Assistant and AI Code Suggestions features, since it both motivates a multimodal, feedback-driven assistant and cautions that most current systems remain single-user (learner-facing) rather than supporting both the presenter/developer and any instructor or reviewer simultaneously.

Borowski, Baliś, and Orzechowski (2024) [4] introduce the Semantic Code Graph (SCG), a source-code information model designed to facilitate software comprehension. The problem addressed is the scarcity of code-representation models that closely track the actual source code while remaining detailed enough to support practical comprehension, refactoring, and quality-monitoring tools. Methodologically, the authors define the SCG as a graph-based model capturing code definitions, declarations, and their relationships, implement extractors for Java and Scala, and provide a protobuf-based intermediate representation together with two open-source tools, *scg-cli* and the *Graph Buddy* IDE plugin. They benchmark the SCG against nine other source-code representation models and empirically evaluate it, alongside the derived Class Collaboration Network (CCN) and Call Graph (CG) models,

on eleven widely used open-source Java and Scala projects, performing tasks such as project-structure comprehension, critical-entity identification, interactive dependency visualization, and similar-code detection through software mining. The results show that SCG, CCN, and CG all exhibit power-law (scale-free) node-degree distributions, and that the additional detail in SCG enables actionable insights, such as identifying excessive mutable-variable usage and locating duplicated methods for refactoring, that are not as readily visible in the coarser CCN and CG models. A limitation is that the empirical validation is qualitative and comparative rather than a large-scale quantitative meta-analysis, and direct extraction support is currently limited to Java, with Scala requiring a separate compiler plugin. Although SCG was not designed for voice interaction, it is relevant to the proposed platform's AI Code Suggestions and AI Error Detection features, since it demonstrates that a structured, dependency-aware representation of code, rather than treating code as plain text, is what enables an assistant to give specific, actionable suggestions such as flagging excessive dependencies or duplicated logic.

Koditala et al. (2024) [5] address the problem of speaker verification (SV) robustness under emotional speech, motivated by the observation that most SV systems are trained predominantly on neutral-toned speech (roughly 90% of typical training data) and consequently exhibit higher false-acceptance and false-rejection rates on emotional utterances. Their methodology uses a CycleGAN-based voice-conversion framework to synthesize angry- and happy-toned utterances from neutral recordings while preserving the original speaker's vocal identity, and augments the training data of an LSTM-based, GE2E-loss SV model with this synthetic data. The reported results show a relative Equal Error Rate (EER) reduction of up to 3.64% on emotional utterances, narrowing the performance gap between neutral and emotional speech, without materially harming performance on genuine (non-synthetic) media speech used as a spoofing-resilience check. A limitation acknowledged by the authors is that adding a large proportion of synthetic angry-tone data degrades performance on neutral utterances, indicating a trade-off rather than an unconditional improvement. This work is relevant to the project's AI-based noise and speaker-focused command recognition, since a user's tone of voice may vary during a live presentation or coding session (e.g., under time pressure or frustration), and this paper indicates that such variability is a genuine, quantifiable source of recognition error rather than a negligible edge case.

Kabir et al. (2021) [6] provide a broad survey of automatic speaker recognition (speaker recognition here being distinct from speech recognition), covering speaker identification, verification, and diarization, along with their text-dependent/text-independent and open-set/closed-set variants. The paper reviews the standard speaker-recognition pipeline (pre-processing, feature extraction, speaker modelling), classical and modern feature-extraction techniques (MFCC, LPC, LPCC, PLP), front-end and back-end modelling approaches (GMM, i-vector, d-vector, x-vector, t-vector, LDA, PLDA), and end-to-end architectures (Deep Speaker, SincNet, RawNet, AM-MobileNet1D), following a PRISMA-based systematic review.

2.1 Comparative Literature Review Table

Reference & Year	Research Focus	Methodology	Key Contribution	Limitation
Deonise et al., 2026 [1]	Speaker-attributed meeting transcription	Pyannote diarization + Whisper ASR + constrained open-weight LLM post-processing, evaluated on AMI Corpus	Conservative, filtered LLM correction preserves WER/cpWER while reducing transcript artifacts; privacy-preserving local inference	Offline only; tested on Headset Mix condition only; no real-time/streaming evaluation
Ahlawat et al., 2025 [2]	Deep-learning-based ASR, monolingual & multilingual	Systematic literature review (2010–2024) of DNN/RNN/CNN ASR models, datasets, toolkits, metrics	Comprehensive taxonomy of ASR architectures; identifies WER trends and low-resource-language gaps	Survey only, no new model proposed; performance figures vary widely by language/dataset
Rodrigues et al., 2025 [3]	AI-driven Intelligent Tutoring Systems in engineering education	PRISMA-based SLR of 46 studies from IEEE Xplore, Scopus, SpringerLink	Identifies interaction-centric vs. model-centric imbalance and dual-user (learner/instructor) gap; recommends multimodal (text, code, voice, diagram) ITS	Only 3 databases; English-only; qualitative synthesis, no meta-analysis of effect sizes
Borowski et al., 2024 [4]	Graph-based source-code representation for software comprehension	SCG model + scg-cli/Graph Buddy tools; benchmarked against 9 models on 11 open-source Java/Scala projects	Structured, dependency-aware code graph yields actionable comprehension insights beyond CCN/Call Graph models	Qualitative/comparative evaluation only; Scala extraction needs a separate compiler plugin
Koditala et al., 2024 [5]	Speaker verification robustness to emotional speech	CycleGAN-based synthetic emotional utterance generation for SV training data augmentation	Up to 3.64% relative EER reduction on emotional utterances via synthetic data augmentation	Improvement on emotional speech trades off against degraded performance on neutral speech at higher augmentation ratios
Kabir et al., 2021 [6]	Speaker recognition (identification, verification, diarization)	PRISMA-based systematic survey of 281 papers on feature extraction and speaker-modelling architectures	Consolidated taxonomy of speaker-recognition techniques and open challenges	Not focused on speech-to-text, voice-command interfaces, or presentation/coding applications

2.2 Research Gap Table

Identified Gap	Observation from Literature	How This Project Addresses It
Speaker attribution for meeting notes	[1] demonstrates that unconstrained LLM-based transcript correction can introduce lexical errors, while constrained/filtered correction is safer	The project's automatic meeting-notes feature should adopt a similarly conservative post-processing approach rather than freely rewriting transcripts
Fragmented capabilities	The six reviewed works each address a single sub-problem (ASR, speaker verification, speaker recognition, meeting-transcription refinement, intelligent tutoring, or code comprehension) rather than an integrated, end-user-facing voice interaction system	VocaSense integrates speech recognition, speaker-aware command handling, noise filtering, presentation control, AI-assisted feedback, and coding assistance into one platform
Multilingual and low-resource performance	[2] explicitly reports higher WER for low-resource and code-switched languages compared to English	VocaSense's multilingual (English, Hindi, Marathi) voice-command support must be evaluated with this known accuracy gap in mind, rather than assuming parity with English performance
Multimodal, dual-user AI assistance	[3] identifies a gap in ITS that combine multimodal interaction (text, code, voice, diagrams) and that support both the primary user and an instructor/reviewer	The AI Presentation Assistant and AI Code Suggestions modules are explicitly voice-plus-visual (slide and code) in nature, moving toward the multimodal direction this review recommends
Structure-aware code assistance	[4] shows that a dependency-aware graph representation of code enables more actionable suggestions than treating code as plain text, but the work is not voice-driven	The Voice-Controlled Coding Module and AI Code Suggestions/Error Detection features can draw on structured code-analysis principles while adding a voice-command interface, which is outside the scope of the cited work
Emotional/acoustic variability in recognition	[5] shows measurable degradation in speaker verification under emotional speech, and [2] notes ASR accuracy is data- and condition-dependent	The system applies AI-based noise filtering and is designed to tolerate natural variation in a live presenter's or developer's speaking tone
Application-level voice UI research	None of the six reviewed papers address voice-controlled slide navigation, voice-driven coding workspaces, or combined presentation-and-coding assistants	This is the specific gap this project's synopsis targets; the literature instead provides the underlying speech, speaker, tutoring, and code-analysis building blocks

3. Problem Statement

VocaSense addresses the challenge that presenters and coding instructors face when relying on keyboards, mice, and remotes for slide control, since these devices disrupt seamless delivery and hinder smooth transitions into live coding, while existing voice-controlled solutions remain limited, lacking multilingual support, intelligent coding assistance, noise handling, and automated meeting documentation . The proposed system overcomes these gaps

through a secure, AI-powered platform unifying voice-controlled presentation navigation, speech-to-text coding, multilingual interaction, and automatic meeting-note generation.

4. Methodology

The proposed system adopts a modular and layered architecture to provide an intelligent platform for voice-controlled presentations and programming demonstrations. The methodology focuses on integrating speech recognition, artificial intelligence, web technologies, and cloud-based services into a unified application that supports hands-free interaction. Each module is designed to perform a specific function while maintaining seamless communication with other modules, resulting in a reliable and user-friendly system. Figure 1 presents the overall system architecture of VocaSense, showing the flow of data from the user through the frontend, voice and AI processing layers, the Spring Boot backend, the presentation/coding/meeting modules, and finally into the database and output layers.

Initially, the user accesses the application through a secure authentication mechanism. New users can register, while existing users log in using their credentials. Authentication is managed using Spring Security and JSON Web Tokens (JWT), ensuring that only authorized users can upload presentations, access meeting records, and use the integrated compiler. After successful authentication, the user is redirected to the dashboard, where all available features are displayed.

The presentation management module enables users to upload PowerPoint files through the web interface. The uploaded presentation is validated and stored securely in the database. Once the file is processed, it is displayed in the presentation viewer, allowing users to navigate slides through voice commands without using a keyboard or mouse. This hands-free approach enhances presentation continuity and allows the presenter to focus entirely on the audience.

The voice recognition module continuously listens for predefined commands using Whisper and the Web Speech API. Before processing the captured speech, the audio signal passes through the AI Noise Filtering module, which removes background disturbances such as environmental sounds and microphone interference. Noise suppression improves speech clarity and increases command recognition accuracy, especially in classrooms, seminar halls, and conference environments.

After noise filtering, the Speech-to-Text engine converts spoken words into textual commands. The command processing module analyzes the recognized text using predefined command patterns and natural language processing techniques. Depending on the detected command, the system performs actions such as Next Slide, Previous Slide, First Slide, Last Slide, Go to Slide, Start Presentation, Stop Presentation, or Open Compiler. If an invalid command is detected, the system prompts the user to repeat the instruction instead of executing an incorrect action.

When the user issues the Open Compiler command, the compiler module is activated.

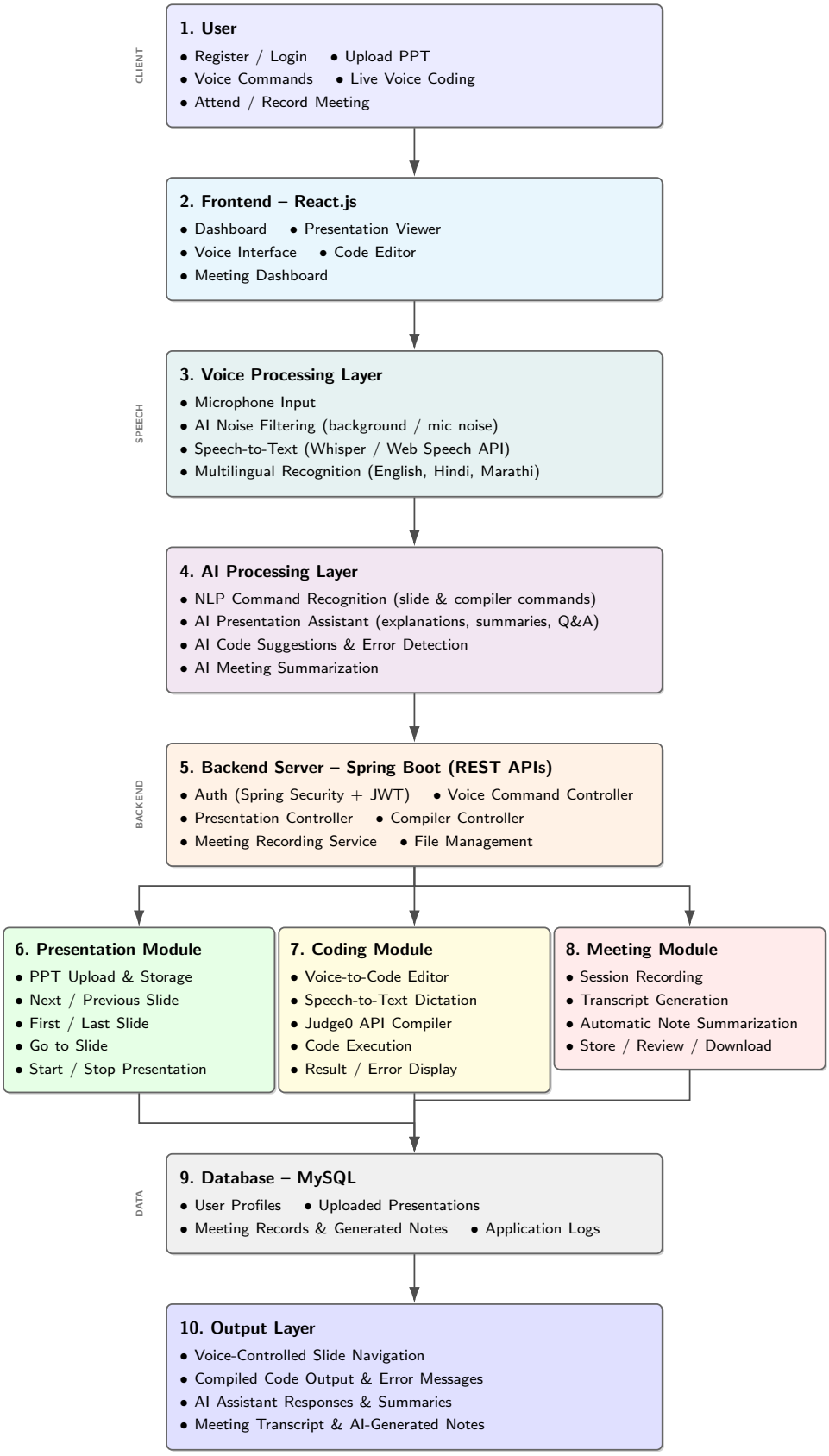

Figure 1: System Architecture of VocaSense

The integrated code editor allows users to dictate source code through speech, which is converted into editable text using speech-to-text technology. Users can review or modify the generated code before execution. The Judge0 API is used to compile and execute the source code, after which the compilation status, execution results, or error messages are displayed within the application. This feature is particularly useful during programming lectures, coding workshops, and technical demonstrations.

The AI Presentation Assistant serves as an intelligent support system during presentations. It assists the presenter by providing concise explanations, generating summaries, suggesting relevant content, and answering presentation-related questions. By integrating artificial intelligence into the presentation workflow, the system reduces manual effort and enables smoother interaction between the presenter and the audience.

To improve accessibility, the proposed system supports multilingual voice commands in English, Hindi, and Marathi. The command recognition module identifies the selected language and interprets voice instructions accordingly. This functionality allows users from diverse linguistic backgrounds to interact naturally with the system, making the platform suitable for educational institutions and organizations operating in multilingual environments.

The meeting management module records presentation sessions with the user's permission. The recorded audio is processed using speech recognition techniques to generate accurate transcripts, which are further summarized into structured meeting notes. These notes can be stored, reviewed, and downloaded later, reducing the need for manual documentation and improving productivity during meetings and lectures.

The backend services developed using Spring Boot coordinate communication between all modules through RESTful APIs. The frontend, developed using React.js, provides an interactive and responsive user interface that enables seamless interaction across different devices. MySQL is used to store user profiles, uploaded presentations, meeting records, generated notes, and application logs. The modular design ensures scalability, allowing future integration of additional AI services, cloud storage, advanced language models, and support for more presentation formats.

The proposed methodology combines secure authentication, intelligent speech processing, voice-controlled presentation management, AI-assisted coding, multilingual interaction, and automated documentation into a single integrated platform. This comprehensive approach improves usability, enhances accessibility, and creates a more efficient presentation environment for students, educators, trainers, and professionals.

5. Project Plan and Timeline

The development of VocaSense follows the official five-month academic schedule mandated by the college, spanning June 2026 to October 2026, and is carried out by a four-member team working in parallel across the majority of activities. The plan is organized into five monthly phases: (i) June – literature survey, research paper study, and require-

ment/feasibility analysis; (ii) July – system architecture, database, and UI/UX design together with the start of frontend and backend development; (iii) August – the core voice, speech-to-text, presentation-control, compiler, and authentication modules; (iv) September – AI-driven modules (presentation assistant, code suggestions, noise filtering, multilingual commands) together with meeting recording/notes and full system integration; and (v) October – testing, bug fixing, documentation, synopsis and thesis writing, and final demonstration/submission. Because several activities are scheduled within the same month, they are carried out concurrently by different members of the team rather than sequentially, which reflects realistic overlap within the constraints of a fixed five-month academic timeline.

5.1 Month-wise Project Schedule Table

Table 1: Month-wise Project Schedule for VocaSense (June 2026–October 2026)

Month	**Activities**
June 2026	1. Literature Survey 2. Research Paper Collection 3. Research Paper Analysis 4. Requirement Gathering 5. Feasibility Study 6. Problem Statement 7. System Analysis
July 2026	8. System Architecture Design 9. Database Design 10. UI/UX Design 11. Frontend Development (React.js) 12. Backend Development (Spring Boot)
August 2026	13. Voice Recognition Module 14. Speech-to-Text Integration 15. PPT Upload & Presentation Control Module 16. Compiler (Judge0 API) Integration 17. User Authentication Module

Month	Activities
September 2026	18. AI Presentation Assistant 19. AI Code Suggestions 20. AI Noise Filtering 21. Multilingual Voice Commands (English, Hindi, Marathi) 22. Meeting Recording & Automatic Meeting Notes 23. System Integration
October 2026	24. Unit Testing 25. Integration Testing 26. Performance Testing 27. Bug Fixing 28. Documentation 29. Synopsis Writing 30. PPT Preparation 31. Final Demonstration 32. Final Submission

6. Expected Outcomes

The proposed system, **VocaSense: Voice Interaction and Coding Assistant**, is expected to provide an intelligent and efficient platform that enables users to deliver presentations and perform coding tasks entirely through voice interaction. By integrating speech recognition, artificial intelligence, and modern web technologies, the system aims to simplify presentation management while improving user convenience and accessibility.

One of the primary expected outcomes is the successful implementation of voice-controlled presentation navigation. Users will be able to upload presentation files and control slides using simple voice commands such as Next Slide, Previous Slide, First Slide, Last Slide, and to Slide, eliminating the dependency on traditional input devices. This hands-free approach is expected to improve presentation flow and allow presenters to interact more naturally with their audience.

The project is also expected to provide a voice-controlled coding environment where users can open an integrated compiler, write source code using speech-to-text technology, compile programs, and view execution results without manual typing. This feature will be particularly useful during programming lectures, coding demonstrations, workshops, and technical training sessions.

The integration of an AI Presentation Assistant is expected to enhance the overall presentation experience by providing contextual guidance, presentation summaries, and intelligent assistance during live sessions. Additionally, the AI-based noise filtering module is expected to improve speech recognition accuracy by minimizing the impact of background noise, re-

sulting in more reliable command execution across different environments.

Another significant outcome is the support for multilingual voice commands in English, Hindi, and Marathi. This capability is expected to make the system more inclusive and user-friendly for individuals with diverse linguistic backgrounds. The meeting recording and automatic note generation features will further assist users by creating organized transcripts and concise summaries, reducing the effort required for manual documentation.

Overall, the proposed system is expected to deliver a secure, scalable, and user-centric platform that combines presentation management, voice interaction, AI assistance, and speech-driven programming within a single application. The developed solution can be effectively used in educational institutions, corporate organizations, technical workshops, online learning platforms, and accessibility-focused environments. Furthermore, the modular architecture provides opportunities for future enhancements, including support for additional languages, advanced AI capabilities, cloud-based collaboration, and integration with Learning Management Systems (LMS), making the platform adaptable to future technological advancements.

References

[1] C.-A. Deonise, T.-M. Coconu, M. K. Z. Bajwa, C. Negru, B.-C. Mocanu, A. Castiglione, and F. Pop, "Speaker-attributed meeting transcription refinement with constrained open-weight language models," *Future Generation Computer Systems*, vol. 185, p. 108648, 2026.

[2] H. Ahlawat, N. Aggarwal, and D. Gupta, "Automatic speech recognition: A survey of deep learning techniques and approaches," *International Journal of Cognitive Computing in Engineering*, vol. 6, pp. 201–237, 2025.

[3] B. Rodrigues, R. Pinto, and G. Gonçalves, "A systematic literature review of ai-driven intelligent tutoring systems in engineering education: Emphasizing personalization, feedback, and student monitoring," *IEEE Access*, vol. 13, pp. 190152–190174, 2025.

[4] K. Borowski, B. Baliś, and T. Orzechowski, "Semantic code graph–an information model to facilitate software comprehension," *IEEE Access*, vol. 12, pp. 27279–27310, 2024.

[5] N. K. Koditala, C. J.-T. Ju, R. Li, M. Jin, A. Chadha, and A. Stolcke, "Improving speaker verification robustness with synthetic emotional utterances," *arXiv preprint arXiv:2412.00319*, 2024.

[6] M. M. Kabir, M. F. Mridha, J. Shin, I. Jahan, and A. Q. Ohi, "A survey of speaker recognition: Fundamental theories, recognition methods and opportunities," *IEEE Access*, vol. 9, pp. 79236–79263, 2021.

(Dr. Sunita Nandgave)	(Dr. Pushpi Rani, Mrs. Shubhangi Ingale)	(Dr. Sarita Patil)
Project Guide	**Project Coordinator**	**HOD**